

Polymarket Trading Bot: A 45-Day Race Against Bugs

*A reflective summary of building, breaking, and measuring a solo trading bot
May 2026 — Companion to the 15-page technical autopsy*

I built a Polymarket trading bot over 45 days on \$1,500 of starting capital. By Day 16, the paper trading fleet showed \$157,115 in apparent profit. **Three measurement bug classes later, the real number was \$1,162** — and that was the discovery that mattered.

At that point I had 180 paper trading bots running on a Frankfurt VPS, all firing on a momentum strategy I'd reverse-engineered from a profitable Polymarket operator (wallet 0x7347, \$107K lifetime PnL). The fleet's apparent PnL was growing exponentially. Every bot looked profitable. I thought I'd cracked it early.

The numbers were lying. I picked one winning window, recomputed the PnL by hand, and the formula was wrong. Polymarket binary contracts have asymmetric payoffs — a 20¢ contract pays \$0.80 on a win, an 80¢ contract pays \$0.20 — and my proxy formula was treating both as if they paid the full cost basis. Once I applied the fix, the fleet flipped from overwhelmingly positive to overwhelmingly negative. **Fourteen days of architectural decisions had been made on inflated numbers.**

Two more bug classes followed. A JOIN bug matching wrong-day windows. And the worst one — a placeholder buried in a depth logger that wrote `yes_mid = 0.5` whenever one side of the order book was empty. My bots read that default as a real mid-price and fired thousands of phantom trades on markets that had already resolved. **The signal had a 99% phantom rate.** Most of my fleet was built around it. When I cleaned the data, paper PnL collapsed from \$157,115 to \$1,162 — a ~135× inflation ratio. The leaderboard had been fabricated, in full, by an empty-book placeholder.

The setup

The project window was April 21 – June 4, 2026, the two months before my college graduation. I picked crypto over weather, sports, and politics because the data was on-chain and the contracts resolved every 5 minutes (288 resolutions per day). The thesis was a calibration arbitrage: Polymarket's binary markets repriced slower than Binance spot, and a bot that detected BTC distance-from-open crossing a threshold could in theory buy the correct side before the Polymarket book caught up.

The pipeline

To find a target operator to reverse-engineer, I built a 3-layer LLM classification pipeline on 2,409 scraped wallets. **Layer 1** used Haiku to bucket every wallet into a rough archetype (~\$8 across the full dataset). **Layer 2** ran a richer 32-column feature pipeline on the 1,100 most interesting wallets, routing between Haiku and Sonnet by confidence (~\$10-12). **Layer 3** used Opus through my Claude Max subscription for deep candidate analysis. Total spend: ~\$30-40, against ~\$150 if I'd run everything on Opus. The tiered design was cost architecture, not just quality optimization — a \$1,500-budget project couldn't have absorbed a \$150 classification bill.

The most useful methodological choice was stratified sampling instead of top-N selection. The ranking formula ($\text{sharpe} \times \sqrt{\text{resolved_markets}}$) buried low-frequency operators by construction. So I sampled across 6 strata of the rank distribution rather than only the top. That's how Brokie surfaced — rank ~1,600 of 2,409, \$453K in positions, a sum-constraint basket strategy the formula had hidden. **The ranking formula was a hypothesis generator masquerading as truth.**

The workflow

I built a multi-Claude workflow that turned solo engineering into something closer to a small team. **I was the Planner**, deciding what to build and how to handle problems. **Claude.ai was the Critic** — analysis, planning, and prompt construction, with explicit behavior rules to push back hard whenever my logic was weak. **Claude Code was the Executor** — running code, debugging, and file operations on the VPS. Shared state lived in three Markdown files (`context.md`, `scope.md`, `CLAUDE.md`) that I kept

current across sessions. Most of the project’s discoveries — including all three bug classes — were caught at the seam between Critic and Executor: results Claude.ai flagged as inconsistent prompted Claude Code investigations that surfaced underlying bugs. **This is how I expect to work in any future role.**

What the clean data showed

After the bugs were patched, I deployed 6 architectures live to Amsterdam VPS with \$300 of real capital. Over ~35 hours, the live fleet produced 105 fills and -\$96 in trading PnL. Across those 6 architectures, **the Spearman rank correlation between paper EV and live EV was -0.43**. Paper EV was inversely predictive of live EV — the bots my paper trading said were best performed the worst live. The pattern was consistent with the bug story: aggressive-firing architectures had picked up more phantom trades, inflating their paper EV with fires that didn’t exist on real order books. **They weren’t profitable. But they weren’t lying either.**

Table 1: Project at a glance.

Metric	Value
Project window	45 days (Apr 21 – Jun 4, 2026), \$1,500 starting capital
Wallets classified / paper bots built	2,409 / 180 (19 operational, 161 pruned)
Paper PnL (pre-fix, contaminated)	+\$157,115 across 399,395 fires
Paper PnL (post-fix, clean)	+\$1,162 across 17,288 fires
Paper inflation ratio	~ 135×
Live deployment	6 bots, 105 fills, -\$96 PnL on \$300 capital
Paper EV vs Live EV (Spearman ρ)	-0.43 (anti-predictive)
Total project cost	~\$388 (\$292 ops + \$96 trading loss)

What I learned

- **Measurement instruments need validation gates separate from the strategies they measure.** My PnL formula looked right at 50¢ entries (the one crossover point where it equals the correct formula), giving it false credibility for 14 days. A single test case at 80¢ would have caught it on Day 1.
- **Default values in shared infrastructure are the most dangerous code you write.** A placeholder `yes_mid = 0.5` passed every gate because the market identifier was correct. Sentinels that look like valid values are the failure mode; obviously-invalid sentinels (None, NaN) force callers to handle them.
- **Stderr cleanliness is not a health signal — observable output is.** Eight bots crash-looped for two hours with completely clean stderr because `sys.exit(1)` produces no error output. Process health requires three signals: running, clean stderr, AND recent output.
- **Methodological rigor doesn’t save you from corrupted data.** Bonferroni correction, minimum-N gates, zone-level comparison — all useless when 80%+ of the underlying data was phantom. The rigor compounded the error instead of catching it.

What I’m taking with me

The strategy produced no detectable edge. I’m not continuing the trading bot. What survives is the durable methodology: the LLM pipeline, the dual-Claude workflow, the bug-class forensic documentation, and the engineering discipline of debugging silent failures. **I’m applying these forward — wherever verification matters as much as inference.**

For the full forensic record across all 180 bots, see the companion 15-page technical autopsy.